UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

SISTEMAS DE INFORMACIÓN I (SI-1) — PROYECTO MRP

GUÍA COMPLETA DE IMPLEMENTACIÓN, FUNCIONAMIENTO Y DEFENSA TÉCNICA

CASO DE USO 12 (CU12):
CONSULTAR EXISTENCIAS DE PRODUCTOS TERMINADOS

Sistema de Información Web para la Panadería **'Santiago'**

Caso de Uso:	CU12 — Consultar existencias de productos terminados
Integrante Responsable:	**Roberto**
Ciclo PUDS:	Ciclo #2 (Abastecimiento, Producción y Existencias)
Rama de Git:	`feature/cu12-consultar-existencias-roberto (100% Local)`
Sub-app Backend:	`backend/apps/productos_inventario/productos_terminados/`
Módulo Frontend:	`frontend/src/apps/dashboard/productos/existencias/`
Fecha de Entrega:	15 de Octubre de 2026
Estado del Caso de Uso:	**100% Desarrollado, Probado y Verificado (0 Errores)**

1. Aclaración Importante: ¿Qué pasó con la Base de Datos?

Es muy normal que al ver mensajes de consola te surjan dudas como: *¿Se crearon bases de datos nuevas? ¿Se modificó la base de datos de mis compañeros?*. A continuación se explica con total transparencia técnica qué ocurrió y por qué tu proyecto está completamente seguro:

A. NO se creó ninguna base de datos nueva permanente ni se alteró PostgreSQL:
La base de datos oficial del proyecto sigue siendo **PostgreSQL 14+** definida por el script maestro Database_Panaderia_Santiago.sql. Las tablas que utiliza tu CU12 (inventario_producto_terminado y movimiento_producto_terminado) ya fueron diseñadas previamente por el equipo.

B. ¿Por qué la consola dijo *'Creating test database for alias default...'*?
Cuando ejecutamos las pruebas automatizadas (manage.py test), Django crea por seguridad una **base de datos temporal desechable** en la memoria RAM de tu computadora. Allí ejecuta las 9 pruebas para comprobar que todo funcione y, en cuanto terminan con éxito, Django la destruye de inmediato (*'Destroying test database...'*). Esto garantiza que las pruebas nunca ensucien ni borren los datos reales de la panadería.

C. Compatibilidad de entorno local sin Docker:
Se agregó en settings.py un ajuste para que, si alguien no tiene Docker o PostgreSQL corriendo en su máquina, pueda probar el backend localmente usando SQLite sin que el sistema falle. Cuando el proyecto se levanta en Docker, automáticamente usa PostgreSQL.

D. Los cambios están ÚNICAMENTE en tu PC:
Todo se encuentra en tu rama local feature/cu12-consultar-existencias-roberto. No se ha ejecutado ningún git push hacia GitHub. Tu equipo en main no ha recibido ninguna alteración.

2. ¿Qué es el Caso de Uso 12 (CU12)?

El **CU12: Consultar existencias de productos terminados** tiene como objetivo permitir al **Propietario** y al **Personal de Ventas** conocer en cualquier momento qué panes están listos para la venta en el mostrador, cuántas piezas hay disponibles de cada uno, cuál es su valorización total en dinero (Bs.) y qué panes están por agotarse o ya se terminaron.

¿Por qué es importante en la Panadería Santiago?
En una panadería tradicional, el personal de mostrador no tiene tiempo de contar panes mientras atiende a los clientes. El CU12 conecta la producción con las ventas: cuando producción termina de hornear (CU10), el stock de pan sube; cuando se atiende en mostrador (CU13), el stock disminuye. El CU12 muestra la verdad de la vitrina en tiempo real con semáforos de advertencia visuales.

3. Mapa Completo de Archivos: Dónde está cada cosa

Todo el código nuevo se ubicó exactamente dentro de las carpetas que corresponden a la arquitectura del proyecto:

Capa	Ruta del Archivo	Estado	¿Qué contiene?
Backend	`backend/apps/productos_inventario/productos_terminados/permissions.py`	**NUEVO**	Control de acceso RBAC. Valida que solo roles con permiso de inventario/ventas puedan consultar.
Backend	`backend/apps/productos_inventario/productos_terminados/serializers.py`	**MODIFICADO**	Serializadores JSON. Contiene la matemática que calcula el valor en Bs. (stock * precio) y clasifica el estado (disponible/bajo/agotado).
Backend	`backend/apps/productos_inventario/productos_terminados/views.py`	**MODIFICADO**	Vistas de la API: ExistenciasListView (lista con filtros), ResumenExistenciasView (tarjetas KPI) y KardexProductoTerminadoView (historial).
Backend	`backend/apps/productos_inventario/productos_terminados/urls.py`	**MODIFICADO**	Rutas de la API bajo /api/productos-terminados/existencias/.

Backend	`backend/apps/productos_inventario/productos_terminados/management/commands/seed_existencias.py`	**NUEVO**	Comando seeder: llena datos realistas de los 12 tipos de pan para probar y demostrar el sistema.
Backend	`backend/apps/productos_inventario/productos_terminados/tests.py`	**MODIFICADO**	9 pruebas unitarias que verifican permisos, cálculos matemáticos, filtros y kardex.
Frontend	`frontend/src/types/existencias.ts`	**NUEVO**	Tipos e interfaces TypeScript para garantizar que los datos no tengan errores.
Frontend	`frontend/src/services/existenciasService.ts`	**NUEVO**	Cliente Axios que pide los datos al backend y tiene datos de respaldo si estás desconectado.
Frontend	`frontend/src/apps/dashboard/productos/existencias/ExistenciasTab.tsx`	**NUEVO**	Pantalla completa con las 4 tarjetas KPI, buscador en vivo, filtros de categoría y tabla con barras de nivel.
Frontend	`frontend/src/apps/dashboard/productos/existencias/components/KardexProductoModal.tsx`	**NUEVO**	Modal emergente que muestra el historial de movimientos de un pan al hacer clic en 'Kardex'.
Frontend	`frontend/src/apps/dashboard/productos/ProductosPage.tsx`	**MODIFICADO**	Página principal donde se agregó la pestaña **'Existencias (CU12)'** junto al catálogo y producción.

4. ¿Cómo Funciona la Solución de Punta a Punta?

A continuación se explica el ciclo de vida completo de una consulta en el sistema, desde que el usuario interactúa hasta que se consulta la base de datos:

1. Autenticación y Autorización (Seguridad RBAC):
El usuario inicia sesión en /login con sus credenciales (por ejemplo, María Elena Gonzales del Personal de Ventas). El backend emite un token JWT que contiene los permisos del rol. Al ingresar a *Productos e Inventario*, el sistema detecta que el usuario tiene el permiso gestionar_inventario y abre automáticamente la pestaña **Existencias (CU12)**.

2. Carga de Métricas en Tiempo Real (KPIs):
El frontend solicita al endpoint GET /api/productos-terminados/existencias/resumen/ los totales globales. El servidor calcula la suma de unidades disponibles, el valor monetario total en bolivianos (Bs.) y cuenta cuántos panes están con stock crítico o agotados.

3. Listado Dinámico con Filtros y Optimización:
El frontend solicita GET /api/productos-terminados/existencias/. La vista en Django utiliza select_related('id_producto__id_categoria') para traer en una única consulta SQL de alta velocidad toda la información del pan, su categoría y su inventario, evitando sobrecargar el servidor.

4. Cálculos Matemáticos en el Servidor:
Para cada pan devuelto, el serializador de Django calcula:
• valor_total_stock = stock_actual * precio_venta (ej. 140 marraquetas a Bs. 0.80 = Bs. 112.00).
• estado_stock: Si tiene 0 unidades es 'agotado'; si tiene entre 1 y 20 es 'critico'; si tiene más de 20 es 'disponible'.

5. Visualización Accesible y Amigable en Pantalla:
El usuario ve la tabla con una barra de progreso que representa la capacidad de la vitrina (de 0 a 150 unidades) y un badge de color intuitivo (verde para normal, amarillo para alerta, rojo para agotado).

6. Auditoría y Kardex Detallado:
Si el usuario desea saber por qué hay cierta cantidad de pan, presiona el botón **'Kardex'**. Se abre un modal interactivo que consulta /api/productos-terminados/existencias/<id>/movimientos/ y muestra cuándo ingresó pan por producción horneada (+) y cuándo salió por ventas (-).

5. Guía Visual de la Pantalla (Lo que ve el Usuario)

Cuando el docente te pida mostrar la pantalla, esto es exactamente lo que tienes y lo que debes señalar:

Elemento en Pantalla	Ubicación	Función / Qué hace
4 Tarjetas KPI Superiores	Parte superior	• **Variedad en vitrina:** Número de tipos de pan activos. • **Stock total listo:** Total de piezas de pan listas en mostrador. • **Valor en mostrador:** Valor monetario total en Bs. a precio de venta. • **Alertas:** Conteo de panes que están por agotarse o ya en 0.
Buscador en Vivo	Barra de herramientas	Permite tipear cualquier nombre (ej. 'marraqueta') y filtra la tabla en tiempo real sin recargar la página.
Selector de Categorías	Filtro desplegable	Permite ver solo panes tradicionales, especiales o dulces.
Filtro de Estado de Stock	Filtro desplegable	Permite filtrar solo los panes 'Normales', solo los 'Bajos (1 a 20 un)' o solo los 'Agotados'.
Barra Visual de Stock	Columna de la tabla	Muestra una barra de color verde, amarilla o roja que indica el porcentaje de llenado de la canasta/vitrina.
Badges Semánticos	Columna de estado	Pastillas de color con texto e icono: 'Disponible' (verde), 'Stock bajo' (amarillo) y 'Agotado' (rojo).

Botón 'Kardex'	Columna acciones	Abre el modal con el historial cronológico de entradas y salidas de ese pan específico.

6. Cómo Probar el Sistema en tu Computadora

Para levantar y probar todo lo que implementaste, sigue estos sencillos pasos en tu terminal de PowerShell:

Paso 1: Activar entorno virtual y poblar datos de prueba:
En la carpeta backend ejecuta el seeder creado para el CU12:

```
cd "d:\Proyecto SI\backend"
.\venv\Scripts\Activate.ps1
python manage.py seed_existencias
```

Paso 2: Correr las pruebas automatizadas (Test suite):
Demuestra que las 9 pruebas de CU12 pasan al 100% en verde:

```
$env:DB_ENGINE="sqlite3"; python manage.py test apps.productos_inventario.productos_terminados.tests
```

Paso 3: Iniciar el servidor backend:

```
python manage.py runserver
```

Paso 4: Iniciar el frontend en otra terminal:

```
cd "d:\Proyecto SI\frontend"
npm run dev
```

Abre tu navegador en http://localhost:5173 e inicia sesión con el usuario de Personal de Ventas: **mgonzales** / contraseña: **Admin123!**.

7. Guión de Exposición Oral para Roberto

Lee y practica este guión para cuando te toque hablar en la presentación ante el docente:

Palabras de apertura:
*«Docente, compañeros, mi nombre es Roberto y me correspondió desarrollar el **Caso de Uso 12: Consultar existencias de productos terminados**. Este caso de uso es fundamental en el ciclo operativo de la panadería porque conecta la producción con las ventas en mostrador. Permite que tanto el Propietario como el Personal de Ventas conozcan en tiempo real el stock físico disponible en vitrina, su valorización monetaria total y las alertas tempranas de reposición.»*

Al mostrar la pantalla:
«Como pueden observar en la pantalla, al ingresar con el rol de Personal de Ventas, el sistema abre directamente la pestaña de Existencias. En la parte superior tenemos 4 tarjetas de métricas clave: total de variedades horneadas, unidades disponibles, el valor total del inventario en bolivianos calculado automáticamente, y los panes que están por agotarse.
En la tabla principal, cada pan tiene su precio unitario, una barra visual de capacidad y un semáforo de disponibilidad. Además, contamos con un botón de Kardex que permite auditar las entradas por producción y las salidas por ventas.»

Respuestas a posibles preguntas del docente:

• **Si el docente pregunta:** *¿Por qué no pusiste un botón para editar el número de stock en esta pantalla?*
Tu respuesta: *«Porque en un sistema de información formal con arquitectura PUDS, el inventario no se altera manualmente con un formulario arbitrario. El stock aumenta de manera atómica cuando el horneador registra la producción (CU10) y disminuye cuando se registra una venta (CU13) o una pérdida (CU18). El CU12 es de supervisión y consulta fidedigna.»*

• **Si el docente pregunta:** *¿Cómo controlaste la seguridad y permisos?*
Tu respuesta: *«Se implementó la clase de permiso PuedeConsultarExistenciasTerminados en Django REST Framework. El backend evalúa si el usuario cuenta con el permiso gestionar_inventario o registrar_ventas. Si un usuario sin permiso o un atacante intenta consumir el endpoint, el servidor rechaza la petición con código HTTP 403 Forbidden.»*

• **Si el docente pregunta:** *¿Cómo trabajaron en Git?*
Tu respuesta: *«Siguiendo la política de nuestro equipo, nadie sube cambios directo a la rama main. Todo mi caso de uso fue desarrollado en mi rama de trabajo feature/cu12-consultar-existencias-roberto con sus pruebas unitarias en verde, garantizando cero conflictos al momento del merge.»*